Modelo Físico do Banco de Dados

UNIFEI | Universidade Federal de Itajubá
Instituto de Ciências Tecnológicas
ECOI2210 – <Análise e Desenvolvimento de Software I>
eduardo.felipe@unifei.edu.br
fabiana.costaguedes@unifei.edu.br
rossana.junqueira@unifei.edu.br
wendelldiniz@unifei.edu.br

Versão: 01/09/2025

Projeto do Banco de Dados

- O projeto de um banco de dados passa por três etapas:
  - Modelagem Conceitual - construção de um modelo conceitual, representado por um diagrama entidade-relacionamento. O modelo conceitual representará os dados que serão armazenados.
  - Modelo Lógico - transformação do modelo conceitual, construído na etapa anterior, em um modelo lógico. O modelo lógico define como o banco de dados será implementado no SGBD de destino.
  - Modelo Físico - gerado a partir do enriquecimento do modelo lógico com detalhes que impactem no desempenho, mas não na funcionalidade. Envolve a definição dos tipos de dados e ajustes para que o modelo lógico funcione com o SGBD escolhido.

O modelo físico do banco de dados

- **Definição**: O modelo físico representa a implementação real do banco de dados em um Sistema de Gerenciamento de Banco de Dados (SGBD) específico. Ele traduz o modelo lógico em estruturas de armazenamento concretas.
- **Analogia**: Se o modelo conceitual equivale à planta baixa de uma casa e o modelo lógico é o projeto detalhado com as medidas, o modelo físico são as especificações de construção, indicando os materiais (tipos de dados), as fundações (tabelas e índices) e como as partes se conectam (chaves estrangeiras).
- Características Principais:
  - Dependente do SGBD (Ex: MySQL, PostgreSQL, SQL Server).
  - Define a estrutura de armazenamento dos dados.
  - Inclui detalhes de otimização de desempenho.
  - É a última etapa antes da criação do banco de dados.

Componentes de modelo físico

- **Tabelas**: Estruturas que armazenam os dados, correspondendo às relações do modelo lógico.
- **Colunas** (Atributos): Campos de uma tabela que armazenam informações específicas. Cada coluna possui:
  - Nome: Identificador da coluna.
  - Tipo de Dado: Define o tipo de informação que pode ser armazenada (ex: VARCHAR, INT, DATE).
  - Restrições (Constraints): Regras que garantem a integridade dos dados (ex: NOT NULL, UNIQUE).
- **Chaves Primárias (Primary Keys - PK)**: Uma ou mais colunas que identificam unicamente cada registro em uma tabela.
- **Chaves Estrangeiras (Foreign Keys - FK)**: Colunas que estabelecem um relacionamento com a chave primária de outra tabela, garantindo a integridade referencial.
- **Índices (Indexes)**: Estruturas que melhoram a velocidade de recuperação de dados.
- **Visões (Views)**: Tabelas virtuais baseadas no resultado de uma consulta SQL.
- **Gatilhos (Triggers)**: Procedimentos que são executados automaticamente em resposta a determinados eventos em uma tabela.

DDL - Data Definition Language

- O que é DDL?
  - É o subconjunto da linguagem SQL usado para criar, modificar e excluir a estrutura dos objetos do banco de dados (como tabelas, índices e restrições).
- Com o DDL, você transforma o diagrama do modelo físico em um banco de dados real e funcional.
- Pense no DDL como o "código de construção" do seu banco de dados.

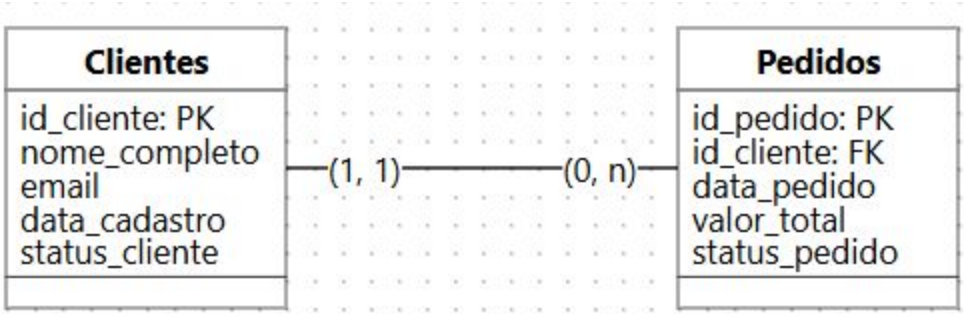
Principais comandos da DDL

- CREATE: Usado para criar objetos.
  - CREATE DATABASE nome_do_banco;
  - CREATE TABLE nome_da_tabela (...);
- ALTER: Usado para modificar a estrutura de um objeto existente.
  - ALTER TABLE nome_da_tabela ADD COLUMN nome_da_coluna TIPO; (Adiciona uma coluna)
  - ALTER TABLE nome_da_tabela DROP COLUMN nome_da_coluna; (Remove uma coluna)
- DROP: Usado para excluir permanentemente um objeto.
  - DROP TABLE nome_da_tabela;
  - DROP DATABASE nome_do_banco;
- **Atenção**: Use com extremo cuidado, pois a operação não pode ser desfeita facilmente, podendo requerer recuperação por backup e perda de dados!

Roteiro para criação do modelo físico

1. **Mapeamento de Relações para Tabelas**: Cada relação do modelo lógico se torna uma tabela no modelo físico.
2. **Mapeamento de Atributos para Colunas**: Cada atributo se torna uma coluna na tabela correspondente.
3. **Definição dos Tipos de Dados**: Escolher o tipo de dado mais apropriado para cada coluna, considerando o SGBD específico.
4. **Definição das Chaves Primárias e Estrangeiras**: Implementar os relacionamentos definidos no modelo lógico.
5. **Aplicação de Restrições**: Adicionar restrições de NOT NULL, UNIQUE, CHECK, etc.
6. **Criação de Índices**: Identificar colunas que serão frequentemente usadas em cláusulas WHERE para otimizar consultas.
7. **Desnormalização (Opcional)**: Em alguns casos, pode ser necessário desnormalizar o modelo para melhorar o desempenho, introduzindo redundância controlada.

Exemplo - modelo lógico



Exemplo - DDL

```
-- Criando a tabela de Clientes
CREATE TABLE Clientes (
	id_cliente INT PRIMARY KEY    AUTO_INCREMENT,
	nome_completo VARCHAR(150) NOT NULL,
	email VARCHAR(100) NOT NULL UNIQUE,
	data_cadastro DATE NOT NULL,
	status_cliente VARCHAR(8) NOT NULL DEFAULT 'Ativo'
);

-- Criando a tabela de Pedidos
CREATE TABLE Pedidos (
	id_pedido INT PRIMARY KEY AUTO_INCREMENT,
	id_cliente INT NOT NULL,
	data_pedido DATETIME NOT NULL,
	valor_total DECIMAL(10, 2) NOT NULL,
	status_pedido VARCHAR(20),
	FOREIGN KEY (id_cliente) REFERENCES Clientes(id_cliente)
);
```

Dicionário de dados

- Cada etapa do projeto de um banco de dados possui um conjunto de ferramentas para sua execução/documentação.
  - Modelo conceitual - diagrama entidade-relacionamento
  - Modelo lógico - mapeamento ER-Relacional
  - Modelo físico - **dicionário de dados**
- O dicionário de dados consiste de uma lista organizada de todos os elementos de dados que são pertinentes para o sistema. Ele fornece informações sobre a definição, a estrutura e a utilização de cada elemento de dados que o sistema utiliza.
- O dicionário de dados documenta a implementação dos dados no SGBD, servindo de consulta para os desenvolvedores de aplicação.

Importância do dicionário de dados

- **Padronização**: Garante que todos na equipe tenham um entendimento comum dos dados.
- **Comunicação**: Facilita a comunicação entre desenvolvedores, analistas de negócios e usuários finais.
- **Manutenção**: Simplifica a manutenção e a evolução do banco de dados.
- **Qualidade dos Dados**: Ajuda a garantir a consistência e a precisão dos dados.

Exemplo - Tabela Clientes

Tabela: `Clientes`
Descrição: Armazena as informações cadastrais dos clientes da empresa.

Nome	Tipo	Tamanho	Valor Padrão	Restrições	Descrição
`id_cliente`	`INT`	-	-	`PK` (Chave Primária), `AUTO_INCREMENT`	Identificador numérico único para cada cliente, gerado automaticamente pelo sistema.
`nome_completo`	`VARCHAR`	150	-	`NOT NULL`	Nome e sobrenome do cliente. O preenchimento é obrigatório.
`email`	`VARCHAR`	100	-	`NOT NULL`, `UNIQUE`	Endereço de e-mail do cliente. É obrigatório e não pode haver dois clientes com o mesmo e-mail.
`data_cadastro`	`DATE`	-	-	`NOT NULL`	Data em que o cliente foi registrado no sistema (formato: AAAA-MM-DD).
`status_cliente`	`VARCHAR`	8	'Ativo'	`NOT NULL`	Situação do cliente no sistema. Aceita "Ativo" ou "Inativo". Se nenhum valor for fornecido, será "Ativo".

Exemplo - Tabela Pedidos

Tabela: `Pedidos`
Descrição: Registra todos os pedidos de compra realizados pelos clientes.

Nome	Tipo	Tamanho	Valor Padrão	Restrições	Descrição
`id_pedido`	`INT`	-	-	`PK` (Chave Primária), `AUTO_INCREMENT`	Identificador numérico único para cada pedido, gerado automaticamente.
`id_cliente`	`INT`	-	-	`NOT NULL`, `FK` (Chave Estrangeira)	Identificador do cliente que realizou o pedido. Deve corresponder a um `id_cliente` válido na tabela `Clientes`.
`data_pedido`	`DATETIME`	-	-	`NOT NULL`	Data e hora exatas em que o pedido foi efetuado pelo cliente.
`valor_total`	`DECIMAL`	10, 2	-	`NOT NULL`	Soma total do valor dos produtos no pedido, com 10 dígitos no total e 2 casas decimais.
`status_pedido`	`VARCHAR`	20	NULL	-	Situação atual do pedido (ex: "Processando", "Enviado", "Entregue", "Cancelado"). Pode ser nulo.

Boas práticas

- Modelo Físico:
  - **Padronização de Nomenclatura**: Use nomes consistentes para tabelas e colunas (ex: tbl_clientes ou clientes).
  - **Comentários no Código SQL**: Documente o propósito de tabelas e colunas complexas diretamente no script de criação.
  - **Otimização Consciente**: Crie índices com critério para não sobrecarregar as operações de escrita.
- Dicionário de Dados:
  - **Linguagem Clara**: Evite jargões técnicos na coluna de descrição.
  - **Mantenha-o Atualizado**: O dicionário de dados deve refletir o estado atual do banco de dados.
  - **Acessibilidade**: Garanta que toda a equipe tenha fácil acesso ao dicionário.

Boas práticas

- DDL:
  - Versionamento de Scripts: Salve seus scripts DDL em um sistema de controle de versão (como Git) para rastrear alterações na estrutura do banco de dados.
  - Scripts de Alteração (Migrations): Para modificar um banco de dados em produção, use scripts ALTER em vez de DROP e CREATE, para não perder os dados existentes.
  - Bancos de desenvolvimento e produção separados: É comum manter uma versão separada do banco de dados para desenvolvimento, de modo que seja possível testar alterações e atualizações em um ambiente controlado, antes de seguir com as mesmas para produção.
- Ferramentas úteis para modelagem de dados e geração de DDL:
  - Muitas ferramentas como MySQL Workbench, pgAdmin e BrModelo permitem que você desenhe o modelo físico e elas geram o script DDL automaticamente.

Documentação SQL dos principais SGBDs

- Oracle - https://docs.oracle.com/en/database
- MySQL - https://dev.mysql.com/doc
- MariaDB - https://mariadb.com/docs
- MS SQL Server - https://learn.microsoft.com/en-us/sql/sql-server/?view=sql-server-ver17
- Postgres - https://www.postgresql.org/docs
- SQLite - https://www.sqlite.org/docs.html